GUJARAT POLICE INNOVATION CHALLENGE 2026

Sentinel

Technical Proposal — High-Level Design (HLD)

Document #2 · Phase 1 Submission

Sentinel — High-Level Design

Submission deliverable per HACKATHON_DETAILS.md §9.2. This document describes the architecture actually implemented in `sentinel-solution/` (verified against real runtime execution, not just design intent — see "Verification status" callouts throughout) plus the documented production path for scaling it to the full ~80,000-camera, 26-department target. Architecture rationale and IN/OUT scoping decisions are recorded in `STRATEGY.md`; this document is the submission-facing HLD derived from it.

1. Solution model and justification

Hybrid architecture: Model 1 (Centralised CCTV Registry & GIS Mapping) — mandatory base — combined with Model 2 (Unified Viewing & Metadata Analytics), with a documented Model 3/4 (VMS federation -> central AI platform) evolution path for statewide scale-out.

This is a hybrid in the sense the problem statement defines (HACKATHON_DETAILS.md §7: "combine elements from two or more models"), and "innovative hybrid/customized architecture with operational value" is an explicit bonus criterion (§10). We claim hybrid credit only for what is genuinely built (Model 1 + Model 2, verified end-to-end — see §5/§9); the Model 3/4 elements are presented as the documented production roadmap (§2, §7), **not** claimed as implemented. Building a real per-vendor VMS federation bus (Model 3) is deliberately out of scope for the build window (see STRATEGY.md's OUT list) — claiming it as built would be exactly the kind of overstatement this submission avoids.

Justification: Gujarat already operates VISWAS/NETRAM/TRINETRA at scale (publicly reported at 7,000+ cameras, Phase-II adding ~10,500 more — figures per Gujarat Home Department public communications, not independently verified by this team). Pitching a from-scratch centralized VMS (Model 4) invites the reasonable jury question "what did you invent that doesn't already exist?" Model 2's direct-connection approach — no new middleware layer, connect straight to each department's RTSP/ONVIF/API surface — is both what the problem statement recommends for teams that don't want to build a federation bus (Model 3) and the most achievable, honestly-scoped target in the build window. The Model 1 base is what makes it more than a point solution: it's the common registry/GIS/ governance index every other model sits on top of (§7 of the problem statement), and it's what a Model 3/4 evolution would extend rather than replace. See `STRATEGY.md` §"The pitch correction" for the full framing.

What is and isn't claimed as the invention (honest differentiation). Detection, ANPR, and cross-camera vehicle tracking are already shipped commercially in India — by Staqu (JARVIS), Videonetics, Vehant, and Innefu, among others (see `../../docs/s■trategy/RESE■ARCH_EXISTIN■G_SYSTEMS.md` §7). This submission does **not** claim to have invented those, and would lose credibility with a jury that knows these vendors if it did. Sentinel's specific contribution is the **open, explainable vehicle-identity-resolution layer**: a persistent vehicle entity assembled from *uncertain* observations — plate-OCR confidence, appearance similarity, temporal feasibility, and recency — where every cross-camera link records *why* it was made (`link_method` / `link_score` / `link_time_gap_s`, §6), and where an unreadable plate degrades gracefully to attribute/appearance tracking rather than dropping the vehicle (§5). The entity-plus-observations-with-explainable-links shape is a lightweight, open-source realization of the pattern intelligence platforms like Palantir Gotham use (Object <-> Track <-> Observation), applied narrowly to vehicles — not a claim to match those platforms' scope.

2. Architecture overview

{

```
26 Departments (incl. existing VMS/TRINETRA/NETRAM)
| ONVIF / RTSP / API
v
```

```
+------------------------------------ ---------------------+
| SENTINEL — pilot/edge tier (this submission, ~30-camera scale)|
| |
| Catalogue client --polls--> cameras.json (camera discovery) |
| | |
| v |
| Stream Manager --opens/closes--> RTSP workers (1 per camera) |
| | PTS-driven, TCP-forced, backoff reconnect |
| v |
| Analytics Pipeline (sampled frames, stride-N): |
| detect (YOLOv8) -> plate region -> enhance -> OCR (PaddleOCR) |
| -> within-camera tracker (temporal plate fusion) |
| -> appearance embedding (Re-ID) |
| -> cross-camera identity resolution |
| | |
| v |
| Event store (PostgreSQL-compatible via SQLAlchemy, SQLite |
| in dev): VehicleEvent, VehicleIdentity, CameraRegistry, |
| WatchlistEntry, Alert, AuditLog, ApiKeyEntry (RBAC) |
| | |
| +--> Watchlist engine -> real-time alerts |
| +--> Purpose-bound, audited vehicle-history API |
| +--> RBAC-gated REST API (FastAPI) |
| | |
| v |
| React frontend: GIS map, live HLS view, |
| movement timeline + explainability, watchlist, |
| gap-analysis / onboarding ops console |
+------------------------------------ ---------------------+
```

Documented production target (not built this week — see §7 and STRATEGY.md's OUT list): a regional media fabric (MediaMTX/go2rtc) and an event bus (Kafka) sit between the per-department connectors and the AI fabric, so ingestion scales horizontally without every analytics worker holding its own RTSP socket per camera.

Core principle stated explicitly, not glossed over: centralize metadata/events/identity/GIS/governance — not every raw video frame. 80,000 cameras x ~2 Mbps ~ 160 Gbps sustained, ~1.7 PB/day. Full central raw-video ingestion is not a credible target for any team at this budget, and we say so rather than implying otherwise. See §7 for the actual scaling strategy (edge inference, selective video egress).

3. Heterogeneous camera / VMS integration approach

- **Discovery, never hardcoding:** every camera URL is derived from the live catalogue (`GET /api/ingest` generically, `cameras.json` for this specific sandbox) — camera IDs and availability can change; the catalogue is the contract (`catalogue.py`).
- **Protocol handling:** RTSP forced over TCP (`OPENCV_FFMPE■G_CAPTURE_OP■TIONS=rtsp_t■ransport;tcp`); mixed H.264/H.265 streams tolerated (decoder warnings at join are logged, not fatal).
- **Timing correctness:** every frame carries `pts_ms` from `CAP_PROP_POS_MSEC`; `CAP_PROP_FPS` is never trusted. Downstream tracking/velocity math uses PTS deltas, never wall-clock arrival time — required because the gateway replays a buffered GOP on connect, which arrives faster than real time.
- **Resilience:** exponential-backoff reconnect (2s -> 30s cap), never a tight loop; a scene discontinuity (the sandbox's recording loop point, detected as PTS going backward) triggers an explicit state reset in the tracker rather than silently producing garbage across the cut.
- **Heterogeneous departments:** `CameraRegist■ry.departmen■t` plus the onboarding API (`POST /cameras`, `POST /cameras/bulk`) let a camera be tagged by owning department independent of protocol/vendor — the registry's job is to be the common index Models 2/3/4 all sit on top of, per HACKATHON_DETAILS.md §7's note

that Model 1 underlies the others.

- **ONVIF as the device-abstraction standard:** the vendor-neutral integration surface targets ONVIF Profile S/T (the standard both Genetec Security Center and Milestone XProtect — 10,000+ device models — build their multi-vendor support on; see `../../docs/s■trategy/RESE■ARCH_EXISTIN■G_SYSTEMS.md` §2). The sandbox exposes plain RTSP, so the pilot connects directly; ONVIF is the documented onboarding path for real heterogeneous department cameras.

- **Private/commercial CCTV coverage (societies, malls, commercial establishments):** the problem statement asks for this "wherever feasible and permitted" — not built or demonstrated this session (no private/commercial feeds exist in the hackathon sandbox to test against), but the same onboarding path above already covers it by design: a private-entity camera is onboarded the same way as a departmental one (`POST /cameras` / `POST /cameras/bulk`, ONVIF/RTSP), tagged with the owning entity in place of `department`, contingent on that entity's explicit consent/access grant — no separate integration layer needed. This is a design-only claim, not a demonstrated one.

- **Cross-camera timing doesn't depend on camera clocks:** within a camera, timing uses PTS deltas (never wall-clock — required for correct velocity/dwell math, see above). But cross-camera correlation (travel-time plausibility between two different cameras' sightings) cannot use each camera's own clock — 80,000 legacy cameras have no guaranteed NTP sync, and assuming otherwise would be exactly the kind of unstated dependency a technical jury would catch. Instead, `VehicleEvent■.observed_at` is stamped once, by the single analytics server's own clock, at the moment each event is persisted (`db/models.py`) — one time authority for cross-camera ordering, independent of any individual camera's internal clock accuracy.

Verification status — CONFIRMED against the live sandbox, 2026-09-04. With real credentials configured, the full chain was tested end-to-end against the actual sandbox, not assumed from an earlier session: catalogue login + refresh (30 real cameras returned, all live, real location names e.g. "01 Chiman bhai Bridge"), real RTSP frame reception via `RtspCameraWorker` (1920x1080, TCP-forced, real PTS), and the **full real analytics pipeline** (YOLOv8 + PaddleOCR + tracker + identity resolution) run against genuine live footage from two cameras: - **cam01 (bridge overview, wide/distant framing):** 4 real vehicles detected in a 25s window with real attributes (colour, type, pixel dimensions, detection confidence 0.41–0.73) — **0 of 4 produced a plate read.** Vehicle crops were small (largest 188x149px), consistent with a wide-angle bridge camera rather than an ANPR-oriented angle. This is the first genuine measurement of the previously-unmeasured real-footage OCR gap (§5) — and it is exactly the case "ANPR failure != tracking failure" is designed for: all 4 vehicles are still logged by attributes, not lost. - **cam12 (Tri Mandir Adalaj toll plaza, closer framing):** 0 vehicles detected in a 30s window — no traffic crossed during the sample (a normal toll-booth gap, not a pipeline failure); one benign HEVC decoder warning ("could not find ref with POC 6") occurred, matching the documented, expected IDR-wait behaviour (§3), not a crash.

Implication, stated honestly: detection and the resilience/timing design are confirmed working on real government footage. Plate-read accuracy on real footage remains genuinely low in this sample (0/4) — not yet sufficient to claim working ANPR on real Indian plates, only that the graceful-failure path works correctly when it doesn't. A larger sample across more cameras/longer windows is needed before claiming a real-footage ANPR accuracy *number* either way — but sandbox connectivity, RTSP resilience, and end-to-end pipeline execution are no longer an open question for this submission.

Root cause narrowed further, 2026-09-04 (ML review): `StubPlateDetector` (`plate_detector.py`) — the localizer actually wired into the live pipeline (`main.py` never overrides `plate_detector=`) — crops a fixed lower-middle third of the *vehicle* bounding box, a geometry that only makes sense for a front/rear-facing ANPR-gantry-style shot. Directly inspected the region it extracts from three real saved sandbox crops (a car and two motorcycles): in all three, the extracted region contains side panel, engine, or seat — never a plate — because these cameras' overview angle never presents the plate-bearing face the heuristic assumes. This means the 0/4 result is very likely a localization-geometry problem, not (or not only) an OCR-quality problem — no amount of PaddleOCR tuning fixes a crop that never contained the plate.

Confirmed at scale and fixed, 2026-09-05. Ran `scripts/real■_anpr_scan.p■y` (no compositing) against all 30 real sandbox cameras, 20s each: 466 real vehicle detections across 14 active cameras, **0 plate reads**, with `StubPlateDetector` still wired — the same failure mode as the N=4 result, now confirmed at N=466. Manually inspected the extracted crop region against the full vehicle image for several detections (e.g. `cam01`'s bus, `cam17`'s

motorcycle): confirmed the stub's fixed-geometry crop misses the real plate every time — on the bus, the actual plate sits at the vehicle's lower-left corner; the heuristic instead extracts the lower-middle wheel/underside region.

Accepted the AGPL-3.0 licensing decision (previously deferred, see below) and integrated `morsetechlab■/yolov11-lic■ense-plate-d■etection` (ONNX weights, `YoloPlateDetector` in `plate_detector.py`, now wired into `main.py` as the default with the same honest-fallback-if-missing pattern as the other models). Re-ran the real trained localizer offline against the 62 saved crops from the same scan: it correctly localizes a genuine plate region on `cam01`'s bus — a real ~25x16px rectangular patch at the same lower-left position identified by manual inspection above, which the stub totally missed — a real localization win. But OCR still reads 0/62: at that resolution/camera distance, the plate region is physically too small for PaddleOCR to read any characters from, independent of localization correctness. Separately, the real localizer is also more conservative than the stub (fires on 19/62 crops vs. the stub's 62/62, since the stub always returns *some* crop) and one of those 19 was a genuine false positive — a "GUJARAT POLICE" signboard on scaffolding that the *vehicle* detector (YOLOv8, COCO classes) misclassified as a vehicle in the first place; the plate regex validator (`PLATE_PATTERN` in `anpr.py`) correctly rejected the resulting non-plate-format OCR candidate, so this false positive never reached identity resolution or a false alert.

Superseded, 2026-09-05 (same day, later): the camera-distance conclusion above was based on an incomplete sample. The prior paragraph inspected only 2–3 of the sandbox's 30 cameras (`cam01`, `cam17`, plus the toll plaza), all wide-overview angles. Following direct guidance from the hackathon organizers (their core team, relayed 2026-09-05: "for demo 1, use any sandbox camera that's clearly visible and test plate detection on it"), reviewed the saved crops from ALL 30 cameras and found **`cam06` is a genuinely close-range, front-facing camera** (an auto-rickshaw shot head-on, filling most of the frame) — categorically different from the wide-overview cameras, and NOT something the earlier "these camera placements are too far/low-res" conclusion accounted for.

Testing `cam06` surfaced a **second, independent, real bug**, this time in `anpr.py` rather than the localizer: the vehicle's plate is a genuine **two-line Indian plate** (e.g. state+RTO code on one line, series+number on the next — common on two/three-wheelers). PaddleOCR correctly detects each line as its own text region ("GJOSA" / "3Y6417" for a real plate), but the code only ever pattern-matched each line INDIVIDUALLY against the full `PLATE_PATTERN` regex — neither line alone can ever match a two-line plate's full format, so a plate a human can read was being silently discarded as unread, independent of localization or camera quality.

Fixed 2026-09-05. `anpr.py`'s `read()` now also concatenates all detected lines (sorted top-to-bottom by bounding-box y-position) into one candidate before pattern-matching, and applies a bounded, symmetric homoglyph correction (`_ambiguous_variants` / `_best_pattern_match`) for OCR's common digit/letter confusions (0<->O, 5<->S, 1<->I, 8<->B, 2<->Z) — found necessary because even the joined string initially read "GJOSAY6417" (0->O, 5->S) instead of a pattern-matching form. **Verified against 13 real frames of the same `cam06` vehicle track:** individual-frame pattern-valid reads went from **0/13 to 8/13** — a real, measured improvement, not a guess. Regression-tested: `tests/test_anpr.py` (5 new tests, pure-Python, no ML deps) plus the full existing 37-test suite and `demo_end_to_end.py`'s synthetic single-line-plate path both still pass.

Still-honest remaining gap, not yet closed: the 8 individual-frame reads aren't fully consistent character-for-character (e.g. "GJ05AAY6417" vs. "GJ05AOY6417" vs. "GJ05AQY6417" — the state+RTO prefix "GJ05A" and number suffix "6417" are stable across most reads, but one letter position varies). `tracker.py`'s `consensus_plate()` does EXACT-STRING majority voting across a track's frames, not per-character voting — with reads split across ~6 distinct near-matching strings, the winning vote share lands around 25%, below `PLATE_MIN_CO■NFIDENCE=0.5`, so the live pipeline would still suppress this specific vehicle's plate as low-confidence rather than accept a partially-wrong read (the intended, safe behavior of that gate — a suppressed read is not a wrong alert). A per-character (or position-weighted) consensus mechanism is the natural next step if more time allows, but is a new mechanism, not implemented this pass — flagged honestly rather than claimed as done.

2026-09-05 (third pass, same day): a real, correct, high-confidence plate read was captured on live government sandbox footage — but see the 2026-09-26 correction below before citing this. Re-ran the full 30-camera scan with both fixes in place (real localizer + multi-line/homoglyph OCR matching), 45s/camera. Result at the time: 687 real vehicle detections, 76 plate localizations, 7 pattern-valid reads — all from `cam06`. Two real vehicles produced these reads: - A car: plate read as `GJ01RP6128` on 4 of 5 consecutive samples (one misread as `GU01RP6128`), confidence 0.85–0.92 each time. The saved vehicle crop showed the plate clearly and legibly at the time. - A second vehicle: `GJ32B5405` / `GO32B5400`, split 1-vote-each (the known consensus-voting gap, still applies here).

The captured votes were fed through the actual production consensus-vote logic (`tracker.py`'s `Track.consen■sus_plate()`): `("GJ01RP6128", confidence=0.81)`, clearing `PLATE_MIN_CO■NFIDENCE=0.5`. **That specific algorithmic check is still valid and locked in as a regression test** (`test_consens■us_plate_rea■l_cam06_vehi■cle_clears_c■onfidence_ga■te`) — given those exact captured votes, the consensus logic correctly accepts them.

> **2026-09-26 correction, then a fresh real result — read this whole block before citing anything above.** Three fresh real scans against the live sandbox (30 cameras twice, plus a focused 3-minute `cam06`-only pass; 928 real vehicle detections, 497 localization attempts total, using the real trained localizer and real PaddleOCR, not a stub) produced 0 pattern-valid plate reads — the `GJ01RP6128` capture above did not reproduce. Two further scans (5 cameras/60s each, then all 30 cameras/40s each — 2,266 more detections) also came up empty until the very last one: **the 40s/camera 30-camera sweep produced 7 pattern-valid reads, 6 of them the same plate on `cam06`: `GJ19BR3000`, confidence 0.917–0.977 across frames 5–30, visually confirmed against the saved crop (`data/anpr_sc■an/20260926T■150303Z/cam0■6/f5_v1_plat■ecrop.jpg`) — a clean, legible front plate on a black SUV, not a false positive.** Total across all 6 runs: 3,670 real vehicle detections, 677 localization attempts, 7 pattern-valid reads. **Use `GJ19BR3000` as the current real-footage headline, not `GJ01RP6128`** — it is a different vehicle at the same camera, reproduced today rather than three weeks ago. See `docs/hackath■on/REQUIREME■NTS_COVERAGE■.md`'s "ANPR-based metadata generation" row for the full detail of all six runs.

Bottom line, corrected: the ANPR pipeline — real YOLOv8 vehicle detection, the real trained plate localizer, real PaddleOCR, pattern validation, and the consensus-vote logic — is verified working end-to-end on live sandbox footage, reproduced 2026-09-26 (`GJ19BR3000`, `cam06`). The earlier `GJ01RP6128` capture (2026-09-05) did not reproduce in 5 attempts before this one succeeded, which is worth stating honestly: a legible plate read depends on real traffic passing close enough to a suitable camera during the sampling window, not a guaranteed per-attempt outcome — do not imply every sampling pass will succeed. `cam06`'s ANPR suitability itself (close-range, front-facing) is confirmed by both the original and this new capture. `EMBEDDING_SI■MILARITY_THR■ESHOLD`/ML-3/ML-4 in `REVIEW_FINDINGS.md`/`CODEX_HANDOF■F_PROMPT.md` remain open for a related but distinct reason — no clean same-vehicle real-footage pairs exist yet to build a held-out Re-ID evaluation from.

Dependency note for whoever installs `requirements-ml.txt` fresh: loading the ONNX plate-localizer weights the first time triggers ultralytics' "requirements: AutoUpdate" behavior, which silently upgrades `onnx`/`onnxruntime` to versions requiring `protobuf>=4` — this **breaks paddlepaddle 2.6.2** at import time (`TypeError: Descriptors cannot be created directly`), reproduced empirically. Fixed by pinning `onnx==1.14.1`/`onnxruntime==1.17.3`/`protobuf==3.20.2` in `requirements-ml.txt`; verified both the localizer and PaddleOCR import successfully together in the same process after the pin.

4. Live stream ingestion / processing / management

Per-camera architecture: one `RtspCameraWorker` thread per live camera, managed by `StreamManager`, which reconciles against the catalogue on an interval — opening workers for newly-live cameras, closing workers for cameras no longer in the catalogue ("pace your load": only open what's being processed).

Frame sampling: `ANALYTICS_FR■AME_STRIDE` (default every 5th frame) bounds inference cost per camera; frame rate and resolution differ per camera and are read from catalogue metadata, not assumed uniform.

Camera health monitoring (Model 1 deliverable): each worker exposes `connected` + `last_frame_at`; a periodic sync loop writes this into `CameraRegist■ry.is_health■y` / `last_seen_live_at`. A camera that has never been confirmed live reports health as `null` (unknown), not a false "healthy" default — an onboarded-but-never-connected camera must not look identical to a confirmed-working one.

Stream health is not analytics health, tracked separately since 2026-09-13: `is_healthy` above only reflects whether frames are arriving over RTSP. Live-reproduced this session: a torch/torchvision ABI mismatch made every analytics call raise while the stream itself stayed connected the whole time, so `is_healthy`/`/health` reported "ok" with all 30 workers up while detection was 100% broken. `StreamManager` now tracks `analytics_degraded` / `last_analyti■cs_success_a■t` at the point `pipeline.process()` is actually invoked (stride-gated, so skipped

frames aren't miscounted as trivial successes), surfaced separately on `GET /cameras` and `/health` rather than folded into the single `is_healthy` boolean.

5. AI video analytics approach

{

```
frame -> vehicle detection -> plate region -> enhancement -> OCR
-> within-camera temporal fusion (majority-vote across sampled frames)
-> appearance embedding
-> cross-camera identity resolution (plate-first, appearance-second)
-> persist -> watchlist check
```

}

Stage	Implementation	Status
Vehicle detection	YOLOv8n (`ultralytics`, pretrained COCO weights)	**Real, verified** against a real photo (car/truck/bus/motorcycle/person classes need no fine-tuning)
Plate localization	Heuristic crop (lower-middle third of vehicle bbox)	Working substitute; no fine-tuned Indian-plate localizer available in the build window — documented gap, not hidden
Plate enhancement	Grayscale + CLAHE + upscale	**Real**, dependency-free, measurably improves OCR hit rate on small crops
OCR	PaddleOCR (pinned 2.9.1 — see §8 for why the pin matters)	**Real, verified** end-to-end including the genuine-failure path (blank plate -> `None`, not a crash)
Within-camera tracking	Real ByteTrack (`ultralytics` `model.track(■tracker="byt■etrack.yaml")`, one persistent tracker instance per camera)	**Real, verified** — Kalman-filter-based association via a library already in our dependency tree, not a new model; IOU-overlap fallback retained for detections ByteTrack hasn't confirmed yet, or the stub detector path
Appearance Re-ID	HSV colour histogram + grayscale template, cosine similarity	**Real, working**, not state-of-the-art (FastReID/OSNet is the documented upgrade path)
Cross-camera identity	Plate-first, appearance-second resolver	**Real, verified end-to-end**: an anonymous (unreadable-plate) sighting at camera A correctly upgrades to a plate identity retroactively the moment ANY camera reads it

Stage	Implementation	Status
Make/model classification	Not implemented	Honest gap — no open-source model classifies Indian-market vehicle make/model well; returns `None` rather than a fabricated guess
Motion attributes (dwell / speed / direction)	Derived from track bbox path over PTS	**Real, verified** — image-plane metrics (px/s, screen-direction), NOT calibrated km/h; enables dwell/wrong-way/stopped-zone filters. Inspired by BriefCam's searchable attributes (§3)`/../docs/s■trategy/RESE■ARCH_EXISTIN■G_`
Low-confidence read suppression	Config threshold on fused plate confidence	**Real, verified** — a shaky read is withheld (vehicle still logged anonymously) rather than spawning a false identity/alert. Inspired by Flock (§1)`/../docs/s■trategy/RESE■ARCH_EXISTIN■G_`
Geo-temporal route reconstruction	Haversine distance between camera GPS + time-gap -> plausibility check; OBSERVED sightings interleaved with honestly-labelled INFERRED camera-less segments (`analytics/geo.py`)	**Real, verified** — the differentiator layer for the "advanced cross-camera correlation" bonus (§10 of the problem statement). Straight-line/lower-bound only; a *high* implied speed flags "not a direct drive" (unmonitored detour / possibly different vehicle). Full road-network (OSM) routing + forward next-camera prediction are documented roadmap, not built — see below

Facial Recognition Systems (FRS) — evaluated, deliberately excluded, not silently omitted. The problem statement lists FRS as an example technology teams may propose. Sentinel does not build it, for two specific, cited reasons rather than a generic privacy objection:

- **Accuracy, cited not assumed.** NIST's FRVT Part 3 demography report (NISTIR 8280) found up to **7,200x variation in false-positive rate** across demographic groups for some algorithms — the error is not uniform, it concentrates. Separately, Delhi Police's own 2018 statement to the Delhi High Court put their deployed FRS system's accuracy at **2%**. Both figures are independently fact-checked citations (see `docs/hackath■on/TASKS.md`'s Model 4 module-doc entry), not claims invented for this document.
- **Legal exposure.** The DPDP Act 2023 exempts law-enforcement processing under §17, but biometric identification at CCTV scale, done wrong, creates the kind of wrongful-identification incident that draws exactly the regulatory and reputational scrutiny a pilot project should not invite before an accuracy/bias evaluation exists.

`analytics/De■liberatelyEx■cludedFaceRe■cognizer` is a real code seam — a Protocol implementation that raises rather than fabricating a face-match result — so the exclusion is enforced in code, not just stated in a document; see the Model 4 pilot slice entry in `docs/hackath■on/TASKS.md`. If a department later mandates FRS with its own validated, bias-audited model, the seam is where it would plug in; nothing here should be read as "FRS is technically impossible to add," only that it was not responsibly buildable in this project's timeline without a validated accuracy/bias baseline.

"ANPR failure != tracking failure" is the core design principle (see STRATEGY.md and `sentinel-sol■ution/README■.md`): a vehicle is never dropped just because a plate can't be read on a given camera. It's logged by colour/type/dimensions/appearance embedding, and the appearance signal carries identity across

cameras until a plate read resolves it. Verified with a real (non-mocked) three-camera scenario: two unreadable plates, one successful read, all three sightings correctly merge into one identity.

Real-world accuracy caveat, stated plainly: OCR accuracy has been verified against clean synthetic plate text (>99% confidence) and against a real photo for vehicle detection — not yet against real Indian-plate photography (motion blur, oblique angles, non-standard fonts/spacing, actual government camera image quality). This is the single most jury-visible number on evaluation day and is currently unmeasured against real footage. Accuracy in production comes from the detector + regex plate-pattern validation + temporal fusion across frames working together, not the OCR engine in isolation.

Vendor-claimed vs. field accuracy — a documented real-world gap, cited not assumed: Re-ID/ANPR benchmark accuracy on lab datasets (Market-1501, DukeMTMC: 90%+) does not transfer directly to field deployment. The clearest recent evidence: Flock Safety — a commercial ANPR network advertising "high-90s" accuracy — was found by Roseville, CA police to have misread **71% of the 1,427 stolen/felony-vehicle alerts** it generated in 2023–24, largely attributed to camera placement/angle rather than the model itself (Business Insider via Gizmodo, Aug 2026; Davis Vanguard, Aug 2026). The lesson taken here is not a specific accuracy number for *this* system (unmeasured — stated above) but the *mechanism*: field conditions (camera angle, distance, placement) degrade accuracy far more than model choice does, and a system must be designed to fail safely when that happens. That is precisely why Re-ID here is a *secondary confirmation signal*, plate-first (§6), and why low-confidence plate reads are suppressed rather than promoted to alerts (§5 table, "Low-confidence read suppression") — a partial or uncertain read is never allowed to fan out into "hundreds of matching vehicles"; it stays an anonymous, attribute-tracked sighting instead.

6. CCTV-to-watchlist correlation and alert methodology

Every persisted `VehicleEvent` with a resolved plate is checked against `WatchlistEntry` (stolen/wanted/blacklisted) at write time (`watchlist/service.py`). A match creates an `Alert` row immediately — polled by the frontend's Alerts panel every 5 seconds.

Governed alert lifecycle (entity-lifecycle pattern, Palantir-inspired — see `../../docs/s■trategy/COMP■ETITIVE_TEAR■DOWN.md`). An alert is not a fire-and-forget log line; it's a live investigation item with an enforced state machine: `new -> acknowledged -> resolved`, with `dismissed` (false positive) reachable from `new`/`acknowledged`. Only transitions declared in `ALERT_TRANSITIONS` (`db/models.py`) are permitted — the API rejects an illegal move with 409 and an unknown state with 400, and the frontend only renders the transitions the server reports as legal, so the governance rule lives in exactly one place. Each transition records **who** made it and **when** (`status_updated_by`/`status_updated_at`), mirroring the purpose-bound audit ethos. `dismissed` being a first-class, auditable terminal is deliberate: marking an AI hit as a false positive is the concrete expression of "AI output is a lead, not evidence" (§6 above) — a recorded human decision, not silence. Verified end-to-end through the real HTTP routes with auth (legal walk, rejected illegal transitions, accountability trail, false-positive path, and backward-compatible `/ack`). The legacy `acknowledged` boolean is kept in sync so nothing that consumed the old shape breaks.

Government-DB integration targets: the watchlist is the local cross-reference; the production integration targets are Gujarat's own law- enforcement databases named in HACKATHON_DETAILS.md §8 — **VAHAN** (vehicle registration), **SARTHI** (licensing), **eGujCop** and **CCTNS** (Crime & Criminal Tracking Network & Systems, the national criminal DB that Indian ICCCs already integrate with — see `../../docs/s■trategy/RESE■ARCH_EXISTIN■G_SYSTEMS.md` §5). The design principle is **query, don't copy**: Sentinel queries these systems for a match on demand rather than replicating their data, keeping it an interoperability layer rather than a shadow database.

"Why was this vehicle linked?" explainability (bonus-relevant, demoable): every `VehicleEvent` records `link_method` (new identity / plate continuation / plate upgrade / appearance match), `link_score` (cosine similarity, when appearance carried the link), and `link_time_gap_s` (seconds since the identity's previous sighting) — captured at the moment of the resolution decision in `identity.py`, because it cannot be reconstructed afterward (`VehicleIdentity` only retains its *latest* embedding). The frontend renders these as a fused, heuristically-weighted score (plate confidence 50% / appearance similarity 35% / recency 15%) — explicitly labeled a transparency aid, not a calibrated probability.

Known failure mode, stated rather than hidden: an appearance-only link (no plate confirmation on either end) can be wrong — two vehicles of the same make/colour genuinely look alike to a colour-histogram embedding. This is exactly

why an appearance-only `link_method` is surfaced differently from a plate-confirmed one in the explainability panel, and why the system's policy (below) treats appearance-only links as leads for an analyst to review, never as a standalone basis for field action.

Operational policy — AI output is a lead, not evidence: no route in this system auto-dispatches an officer or auto-confirms an identity. Every watchlist hit lands in the Alerts panel for human acknowledgement (`routes_alerts.py`); every vehicle-history/search result carries its `link_score` so an investigator sees *why* the system linked a sighting, not just that it did. This is also the concrete mitigation for **automation bias** (an operator rubber-stamping a high-volume alert feed without scrutiny): low-confidence links are visually distinguished, not buried at the same weight as a plate-confirmed one, so the operator's attention is drawn to exactly the cases that need it.

7. Scalability toward ~80,000 cameras

See the standalone `SCALABILITY.md` for the full plan-for-scale deliverable (compute sizing, storage tiers, bandwidth math, cost estimate, HA/DR). This HLD states the architectural shape only:

- **Edge inference, not central raw video.** Detection/OCR/tracking run near the camera (regional edge compute), emitting only structured events (plate, timestamp, camera, embedding vector) to the central plane — matches the ~160 Gbps / ~1.7 PB/day math that rules out full central video ingestion.
- **Horizontal scaling by adding stream-manager instances**, each owning a shard of cameras, publishing events to a shared bus (Kafka/RabbitMQ — documented, not deployed this week) rather than one process holding every RTSP socket.
- **PostgreSQL + PostGIS** (SQLite in this dev build, swappable via `DATABASE_URL` since all access goes through SQLAlchemy) for the event/ registry/identity store at production scale, partitioned by time and region.
- **RBAC is designed to extend, not rebuild:** the current API-key + role model (`api/auth.py`) maps directly onto department-scoped roles at scale — `ApiKeyEntry.■department` already exists for this.

8. Technologies, frameworks, tools

FastAPI, SQLAlchemy (PostgreSQL-ready), OpenCV, YOLOv8 (`ultralytics`), PaddleOCR 2.9.1 (pinned — **PaddleOCR >=3.0 changed its public API in a way that breaks this project's OCR wrapper; verified by installing 3.7.0 and hitting the incompatibility at runtime**, not assumed), React, Leaflet, hls.js. All open source, per the hackathon's Open Source Requirement.

Operational note verified this build: on Windows, constructing the YOLOv8 (torch-backed) detector before the PaddleOCR (paddle-backed) reader in the same process is required — the reverse order corrupts torch's native DLL loading. Documented and enforced in `main.py`; only verified on Windows, not yet on the likely-Linux production/demo box.

9. Security, privacy, RBAC

- **RBAC is real, not a stub:** every API route requires an `X-Sentinel-API-Key` header mapping to a role (`viewer` / `investigator` / `admin`) via `api/auth.py`. A request without a valid key is rejected (401/403), not logged-and-allowed. This is a genuine, if minimal (no full OAuth/session flow — an honest scope choice for the build window), implementation of Model 1's explicit "department-wise RBAC" deliverable.
- **Department scoping is deliberately NOT applied to vehicle search/ movement-history queries — stated explicitly here, not just in a code comment (flagged by an independent project review, 2026-09-11, as the kind of tradeoff that needs a named justification in the submission docs, not just `api/auth.py`'s docstring).** `GET /cameras` and its siblings ARE department-scoped (an investigator tied to one department sees only that department's cameras); `GET /vehicle/{pl■ate}/history` and `/vehicle/sea■rch-by-attri■butes` are NOT — any `investigator`-role key, regardless of which department it's tied to, can trace any plate across every

department's cameras. This is a deliberate design choice, not an oversight: cross-department vehicle-identity correlation is this project's stated core differentiator (§1), and department-scoping the one capability that makes that possible would defeat it. The tradeoff is real, though — it means "investigator" effectively means "investigator, all departments" for the single most sensitive query surface in the system. A production deployment should weigh whether a narrower "cross-department escalation" role, or an additional audit flag on cross-department results, is warranted; not built here.

- **Object-level department scoping closed on the remaining endpoints (2026-09-13):** an independent review found the HLS stream (playlist/ segment), camera last-detection, vehicle-crop, and alert list/status/ ack endpoints were missing the object-level `department_scope()` check that `GET /cameras` already had — a viewer-role key from one department could reach another department's stream/crop/alert by ID even though the list view correctly hid it. Fixed with the same pattern used elsewhere: hard-block (404) for `viewer` outside the owning department, audit-and-allow for `investigator` (logged to `AuditLog`), matching the deliberate cross-department design for vehicle search described above.
- **Purpose-bound, audited queries:** every vehicle-history/search query logs the *authenticated* user (from the API key, not a self-reported string), stated purpose, and optional case ID to `AuditLog` — this closes a real gap found during this build: an earlier version trusted a client-supplied `user_id` string, which made the audit trail spoofable.
- **Enforced data retention (DPDP storage limitation):** vehicle events, identities, and alerts are automatically purged after a configurable window (`db/retention.py`, default 30 days pilot); **audit logs are kept longer** (default 365 days) so the accountability trail outlives the personal data it describes. Modelled on the UK National ANPR system's enforced 12-month schedule — and on the *criticism* that system drew for lacking enforced oversight (see `../../docs/s■trategy/RESE■ARCH_EXISTIN■G_SYSTEMS.md` §6): we enforce retention in code, not policy-on-paper. Admins can view the policy (`GET /admin/reten■tion-policy`) and trigger a purge on demand (`POST /admin/purge`) so the control is demonstrable, not just claimed. Known gap, stated: per-case evidential retention exemption (CPIA-style) is not built — export to a case file before data ages out.
- **DPDP posture:** Section 17 of the DPDP Act 2023 exempts law-enforcement processing, but this system is designed as if the Act's substantive obligations apply anyway — purpose limitation, audit logging, retention limits (above), and data minimization (attribute-only fallback instead of storing raw video) are all load-bearing design choices, not decoration.
- **Purpose limitation, enforced not just stated:** every deployment is configured for a stated use case (e.g. vehicle-tracking for an active case), and every history/search query requires a `purpose` string that is itself audit-logged (§9 above, `AuditLog`) — the system does not expose a generic "browse all camera activity" capability; querying requires a plate, a case, or a watchlist match to start from.
- **Right to erasure / breach notification, stated honestly as unbuilt:** the DPDP Act's erasure and breach-notification obligations are not yet implemented as self-service flows (no "request my data be deleted" API, no breach-notification pipeline). Retention enforcement (`db/retention.py`) is the mechanism that would carry this — a purge trigger and an admin notification hook are the concrete next steps, not built this week.
- **Bias evaluation:** no demographic-parity testing has been run against the detector/Re-ID models — there is no vetted Indian demographic benchmark available in the build window to run it against honestly. Stated as a pre-deployment requirement, not deferred silently: models should be evaluated for performance parity across vehicle types/regions before operational use, with periodic re-audits as the fleet/dataset changes.
- **Anticipated adversarial framing, addressed directly:** a privacy advocate's strongest objection is that attribute + appearance tracking is "facial recognition without the face" — a movement record built without individual consent. The honest answer is not that this concern is wrong, but that the design deliberately narrows it: no face embeddings are computed or stored (§5, `attributes.py` returns no face/identity biometric), every query is purpose-bound and audited (this section), retention is enforced in code with a short default window (30 days) rather than promised in policy, and the system explicitly does not claim to identify who someone *is* — only what vehicle was seen where, which is the narrower, DPDP-Section-17-scoped claim this proposal makes throughout.
- **Security frameworks:** the design aligns with NIST Cybersecurity Framework and CISA Secure-by-Design principles (the same frameworks commercial ANPR networks like Flock cite) — hashed credentials, least-privilege RBAC, audit logging, and input validation (the SSRF fix below) are the concrete expressions of that alignment, not a compliance checkbox.
- **Found and fixed during this build, not shipped:** an SSRF vulnerability in the HLS proxy (`routes_stream.py`) where a client-controlled segment URL could make the backend fetch arbitrary hosts using the sandbox's

authenticated session — closed with a same-host validation check, verified against both the attack case (rejected) and the legitimate same-host case (still works).

- **API keys are stored hashed** (SHA-256), never in plaintext, so a database leak doesn't hand out working credentials.

9a. Evidence integrity and §63-oriented export (added 2026-09-19)

Electronic evidence is only as good as the provenance travelling with it, so each stored detection crop carries an integrity and provenance record created **at write time**, not at export time.

- **Hash at capture.** `analytics/ev■idence_integ■rity.py` computes the crop's SHA-256 immediately after `cv2.imwrite` returns, and `pipeline.py` persists it with the timestamp it was taken at (`vehicle_even■t.crop_sha25■6` / `crop_hashed_at`) in the same commit as the file path. It is never recomputed into that column afterwards. A hash refreshed at export time would prove only that a file matches itself a moment ago — it would say nothing about the bytes between capture and export, which is the entire question.
- **Verification reports, it does not heal.** `verify_crop()` recomputes the hash of the file as it exists now and compares: `VERIFIED` / `HASH_MISMATCH` / `FILE_MISSING` / `NO_HASH_RECORDED`. A mismatch is surfaced with *both* hashes rather than silently overwriting the stored value. Rows predating this feature report `NO_HASH_RECORDED` — absence of a hash is not evidence of tampering, it means the guarantee was never established, and conflating the two in either direction would be a lie.
- **Production method is recorded, including the real-vs-stub seam.** The stamp captures the concrete detector / plate-detector / OCR / Re-ID *class names* in force, not just the configured weight filenames: a `StubPlateDetector` and a `YoloPlateDetector` run identically as far as config is concerned and produce completely different artefacts. A package that cannot tell a reviewer which one ran is not a provenance record. `build_rev` comes from `SENTINEL_BUILD_REV` and reads `unknown` when the deployment does not supply it, rather than inventing a version the code cannot verify about itself.
- **Export is tier-gated, purpose-bound and audited.** `api/routes_e■vidence.py` serves the package as JSON and as a printable Part A / Part B certificate. Both require investigator+, require a stated purpose, and are gated to `CONFIRMED` via the existing `evidence_cla■ss.EXPORTABL■E_CLASSES` — `PROBABLE` is refused too, since an export is the point at which a hedge stops being visible. Every export writes an audit row *before* the artefact is produced, so an attempted export is on the record even if rendering fails. The PDF renders the same package the JSON returns rather than recomputing it, so the two cannot disagree about one artefact.
- **Verification is deliberately not tier-gated.** Checking whether an image was altered is an integrity question, not an evidentiary claim, and an investigator assessing a `LEAD_ONLY` lead is entitled to the answer. Only *export* is restricted.

Status: **AUTOMATED-TEST VERIFIED** — 27 tests, including one that drives the real `AnalyticsPipeline` and asserts the stored hash matches an independently computed hash of the file on disk, one that edits the file after the write and asserts `HASH_MISMATCH`, and seven HTTP-layer tests that are the only ones actually exercising the purpose floor and the role gate. Not field-verified: never run against the real accumulated `sentinel.db`, no frontend control is wired, and the wording is "§63-oriented" pending legal review (see §10).

10. Known gaps (stated, not hidden)

- WHEP (low-latency WebRTC live preview) is not implemented — HLS via an authenticated backend proxy is the working live-view path.
- **Camera ANPR-suitability geometry is NULL for every event written before 2026-09-19**, and the plate-region measurement produced by the default `StubPlateDetector` is an *upper bound* on the plate, not the plate: it can prove a camera geometrically incapable of ANPR, but it can never establish that one is capable (`anpr_suitability.py` reports that case as `INCONCLUSIVE`). The >=120 px width figure the verdict is stated against is a **vendor claim** convergent across four vendors, not a threshold this project measured. Blur, glare, view angle, lens and distance are not measured anywhere and are listed as unmeasured on the screen rather than

silently omitted.

- **The §63-oriented evidence export has had no legal review and no run against the real accumulated database.** A UI control now exists (`EvidenceExpo■rtView.jsx`, wired 2026-09-19, exercised over real HTTP: 409 for LEAD_ONLY, 422 for a short purpose, a real PDF for CONFIRMED); the legal and real-data gaps are unchanged. The mechanism is built and tested (§9a), and the wording throughout is deliberately "§63-oriented" / "prepared to support review under Section 63", never "§63 compliant" or "court admissible". Whether a package produced this way is admissible depends on the operating procedure around the system and on qualified legal counsel — both outside this codebase. The Part A / Part B declarations are printed as blank fields for a human to complete and sign; nothing auto-signs them.
- ByteTrack (ai fabric bonus criterion) **is now implemented** via Ultralytics' built-in tracker (`model.track(..., tracker="byt■etrack.yaml"■)`), one persistent instance per camera — verified end-to-end (stable track IDs across repeated frames, correct per-camera isolation, correct reset on scene discontinuity; see commit history). The custom IOU-overlap tracker remains as the fallback when no external track_id is available. Not yet tuned: ByteTrack's `track_buffer` (default ~30 calls) counts *sampled* frames (every `ANALYTICS_FR■AME_STRIDE`th), not raw frames or elapsed time, so its occlusion-tolerance window is decoupled from real time in a way the PTS-driven `TRACK_TIMEOUT_MS` elsewhere in the pipeline is not — worth revisiting if occlusion handling needs tuning against real footage.
- Real Indian-plate OCR accuracy is unvalidated against actual government footage (see §5).
- **Real sandbox onboarding completed 2026-09-04.** All 30 real cameras registered via `scripts/onbo■ard_from_cat■alogue.py` against the live running backend. Department inference genuinely failed on real data (28/30 UNASSIGNED — real camera names are place names like "Chiman bhai Bridge," not department names; the keyword heuristic's premise didn't hold once tested against real data, not a bug to fix). GIS coordinates: added an honest area-centroid lookup (city/locality-level, NOT measured camera GPS, clearly labeled as such) — 20/30 cameras plotted, 10 deliberately left un-plotted rather than guessed (ambiguous village names with no confident reference). `GET /cameras/gap■-analysis` verified live: 30/30 registered, 0 live-not-onboarded, 12/30 showing real "unhealthy" status — genuine real-world connectivity flakiness across government camera infrastructure (expected per this document's own framing, not a defect), with Model 1's health-monitoring deliverable correctly detecting it live.
- ~~Sandbox RTSP validation could not be reconfirmed~~ — **confirmed 2026-09-04** against the live sandbox (30 real cameras, real frames, full pipeline run against genuine footage from cam01 and cam12; see §3). Real-footage plate-read accuracy is now a measured (not just stated) gap: 0/4 in the tested sample — re-test against more cameras before the live demo to get a larger sample, not to re-establish basic connectivity.
- Legacy-fleet credential management (vendor password rotation, mixed Basic/Digest/proprietary-token auth across 26 departments' real cameras) is not addressed — this pilot connects to one sandbox with one access token. Real onboarding needs a per-camera/per-department credential store with rotation support; not designed or built this week.
- Re-ID similarity search at full scale (matching one query embedding against a historical store spanning 80,000 cameras) is not a solved problem here or in the wider Re-ID literature — see `SCALABILITY.md` §2 for the documented approach (candidate filtering before similarity search, not a brute-force global nearest-neighbor scan).
- **Identity-resolution geo-feasibility gate — real bug found and fixed against continuous live sandbox traffic (2026-09-04).** Appearance-only cross-camera matching (`analytics/id■entity.py`) previously scored every candidate identity in the 15-minute association window by cosine similarity alone, with no check on whether the candidate's last-known camera was physically reachable. Under sustained real sandbox traffic (not synthetic data) this let one identity drift across ~100 unrelated real vehicles statewide within 15 minutes on appearance alone — any sufficiently common colour/shape matched regardless of distance, surfacing exactly the "same appearance, different vehicle" failure mode this document already named in §6, but only now actually triggered by volume. Fixed: every appearance-match candidate is now also run through the same haversine-distance / plausible-speed physics as the forward-looking `rank_candida■te_cameras` feasibility filter — an infeasible candidate is excluded outright, not just deprioritised (same "ELIMINATED not just deprioritised" semantics as geo.py). Verified two ways: (a) a standalone runtime test reproducing the exact original failure shape (400km/2min -> correctly rejected into a new identity; a few km/2min -> correctly linked), and (b) the full `demo_end_to_end.py` integration test (real YOLOv8 + PaddleOCR + identity resolver, not mocks) still passes all 5 evaluation checks with the gate active. Fixing this also surfaced two secondary bugs, both fixed the same pass: the fixed 15-minute association window was now the thing

incorrectly rejecting a genuinely plausible match (a 35-minute, 44 km/h hop between two real cameras) that the gate itself would have accepted — widened to 60 minutes now that the gate, not the window, does the actual physics check; and `AnalyticsPipeline` had no injectable clock, so a fast synthetic test run and its persisted `VehicleEvent■.observed_at` disagreed about elapsed time, which the Vehicle Intelligence timeline's own "inferred segment" display caught (nonsense speeds like ">=59,832 km/h — not a direct drive" on a sighting the resolver had just correctly linked) — fixed via a `clock` constructor param, defaulting to real wall-clock time for production, overridable for deterministic testing.

- **Geo-temporal layer — road-network routing + predictive pruning are roadmap, not built.** What's built (`analytics/geo.py`) is straight-line (haversine) distance + time feasibility for the INFERRED route segments — honest and dependency-free. NOT built: (a) true road-network / OSM travel-time (OSM coverage of Gujarat's minor roads is incomplete — an external-data dependency, documented rather than faked), and (b) forward "PREDICTED next-camera" candidate pruning to constrain the expensive cross-camera match. At the ~50-camera pilot, pruning saves nothing observable (brute-force is instant); its value is the 80k scalability story (`SCALABILITY.md` §2's candidate-filter-before-vector-search). If built, pruning must be a soft ranking hint, never a hard exclusion, and must never override a confirmed plate read — a false negative (losing the vehicle) is worse than extra compute for a police tool.
- **`ultralytics` (the vehicle detector library, `analytics/de■tector.py`) is ALSO AGPL-3.0-licensed — flagged 2026-09-11 by an independent project review, previously undisclosed here.** The plate-localizer's AGPL-3.0 risk below was flagged and accepted, but `ultralytics` itself (used for vehicle detection AND the within-camera ByteTrack tracker, not just the plate-detection seam) carries the same license and the same network-use disclosure obligation for a government backend serving predictions over HTTP — this is the same accepted risk category, now disclosed for both components rather than only one. No different decision is implied; a real production deployment's legal review (already recommended below) should cover both dependencies, not just the plate localizer.
- **Fine-tuned plate localizer — evaluated, AGPL-3.0 risk accepted and integrated 2026-09-05.** `morsetechlab■/yolov11-lic■ense-plate-d■etection` (Hugging Face) is a real, named-publisher YOLOv11 plate detector shipping both `.pt` and ONNX weights — ONNX avoids the arbitrary-code-execution risk of loading an unvetted `.pt` file, which is why an earlier candidate from a single-contributor repo was rejected (re-checked 2026-09-05: `Koushim/yolo■v8-license-p■late-detecti■on` is MIT-licensed but ships only a `.pt` file from a single contributor — same rejected risk profile despite the better license; `keremberke/y■olov5s-licen■se-plate` has no stated license at all — neither clears the bar `morsetechlab` does). This one is licensed **AGPL-3.0**, a copyleft license whose network-use clause has real implications for a government backend — **explicitly flagged to and accepted by the project owner** as a documented risk, not a default choice; a real production deployment should get a legal review of this clause before shipping, not just this note. Its own model card also discloses train/test contamination in its reported accuracy, so the number (mAP@50 0.98) is stated by the publisher as unreliable — verified independently against real sandbox footage instead (§3 above): it correctly localizes a real plate the previous heuristic crop missed, but real-footage OCR read accuracy is still 0/62 at these cameras' resolution/ distance, a separate limitation from localization correctness.
- **IISc UVH-26 — a stronger Indian-specific detector, evaluated, not swapped in.** AI for Integrated Mobility @ IISc released `iisc-aim/UVH-26` (Hugging Face): YOLOv11-S/X, RT-DETRv2-S/X, and DAMO-YOLO-T/L weights trained on 26,646 real Indian urban-traffic images (~2,800 Bengaluru Safe-City cameras, 1.8M boxes, 14 Indian vehicle classes), backed by an arXiv technical report (2511.02563). Reports up to 31.5% higher mAP than COCO-pretrained baselines on their own benchmark — a relative figure under their evaluation, not a claim about accuracy on our footage. This is real, institutionally credible, and directly relevant (our current detector is COCO-pretrained YOLOv8n, not Indian-traffic-specific). Not swapped in this build: doing so properly means re-validating detection end-to-end against real footage before trusting it in front of a jury, which this build window didn't have room for after the ByteTrack work above. Documented as the strongest available upgrade path for the detector stage specifically.
- **FastReID — the documented Re-ID upgrade, unchanged assessment.** `JDAI-CV/fast-reid` (GitHub) is a real, mature Re-ID research framework covering vehicle Re-ID specifically, confirmed still the right target if `ColorHistogr■amEncoder` (`reid.py`) needs to be replaced — consistent with what STRATEGY.md already named before this research pass. Hugging Face's own vehicle-Re-ID model listings are sparse and not vetted enough to trust over a maintained framework — a single downloaded checkpoint from an unvalidated source was correctly not treated as a shortcut here.

- **VLM-based OCR — noted as a production alternative, not attempted.** General-purpose vision-language models (Qwen2.5-VL, LLaVA, moondream — run locally via Ollama or similar) can perform plate OCR as a byproduct of broader visual understanding, and are reportedly more robust to font/script variation than a fine-tuned CNN — potentially relevant given Indian plates' format diversity. Not pursued this build: it requires standing up a new local-LLM-serving runtime competing with YOLO/PaddleOCR for the same GPU, adding latency and a new infrastructure dependency — exactly the kind of new architectural layer out of scope for this window (see STRATEGY.md).

11. Technical prerequisites and information required from participating departments

Onboarding a department's cameras into Sentinel does not require replacing that department's VMS or granting Sentinel any control over it — this section states plainly what is actually needed, since the problem statement asks for this explicitly and it was previously implicit rather than written down in one place.

Required from a department, per camera or per camera group:

- A reachable stream URL and protocol: an RTSP URL (`rtsp://host:■port/path`) or, preferably, ONVIF discoverability (Profile S/T) on the department's network segment (§3). If only a proprietary VMS API/SDK is exposed (no direct RTSP/ONVIF), the department needs to confirm whether that API can issue a stream URL or export events — a case-by-case integration, not assumed to exist.
- Network reachability from wherever the Sentinel ingestion point runs to that camera/VMS — a routed path or an explicitly permitted firewall rule, since Sentinel does not tunnel through NAT/CGNAT on its own.
- Credentials scoped to viewing/streaming only (least privilege) — not admin/configuration access to the camera or VMS. Sentinel never needs write access to a department's own system.
- Camera identity and metadata: a stable camera ID, install location (address and, ideally, GPS coordinates — §3's GIS layer and §5's geo-temporal correlation both depend on this being accurate, not inferred), owning department, and camera type (fixed/PTZ/dome).
- Make/model and resolution, if known — feeds directly into the camera ANPR-suitability scoring (`GET /cameras/anp■r-readiness`, §9a) rather than being guessed.
- For a private/commercial camera (society, mall, business — §3): the same technical fields above, plus the entity's explicit written consent to onboard, since Sentinel enforces purpose-bound, audited access and will not onboard a feed without a named responsible owner.
- If federation-style correlation with another department's own VMS is desired (Model 3, §1): that VMS's own event/metadata export mechanism (API, webhook, or batch export) — Sentinel's `VMSAdapter` pattern (`analytics/fe■deration.py`) is the integration point, but each adapter is written against a specific real system, not a generic "any VMS" claim.

Explicitly NOT required from a department:

- No requirement to migrate off, reconfigure, or expose their existing VMS's storage/recording/playback functions — Sentinel connects alongside, not in place of, per §1/§2's direct-connect principle. Existing systems (VISWAS/NETRAM/TRINETRA or any departmental VMS) keep operating unchanged.
- No requirement for cloud/internet connectivity if the department's network is on-prem/air-gapped — the ingestion point can be deployed within the department's own network boundary; only the analytics output (structured events, not raw video, per §7) needs to reach the central plane.
- No requirement to standardize on ONVIF if RTSP is already available — ONVIF is the preferred discovery path (§3) but not mandatory.

Open, not yet resolved: legacy-fleet credential rotation across 26 departments' real cameras (mixed Basic/Digest/proprietary-token auth) is not addressed by this pilot, which connects to one sandbox with one access token (§10) — a real statewide rollout needs a per-camera/ per-department credential store with rotation support, not designed or built this week.

12. Risk register

Risk	Likelihood	Impact	Mitigation
Real Indian-plate OCR accuracy is materially worse than synthetic-test results	High	High — undermines the core demo claim	Temporal fusion + regex validation + low-confidence suppression already reduce reliance on any single read (§5); re-test against real sandbox footage before the live demo, not after
Appearance-only Re-ID false match (two similar vehicles)	Medium-High	Medium — wrong lead, not wrong action	Plate-first resolution, appearance as secondary signal only, explainability panel flags appearance-only links distinctly (§6)
Sandbox access token unavailable/expired before the demo	Medium	High — blocks live verification entirely	Confirm access early, not on demo day; stub fallback path already exists and runs (loud warning, no fabricated detections) if it fails
Operator alert fatigue / automation bias on a live feed	Medium	Medium — undermines the human-in-the-loop safeguard's real value	Low-confidence links visually distinguished, not equal-weighted with confirmed ones (§6); no route auto-dispatches on a match
Windows-specific torch/paddle DLL conflict resurfaces on the demo machine	Low-Medium	High — blocks the whole pipeline from starting	Load order is enforced in `main.py` and documented (§8); verify on the actual demo hardware beforehand, not assumed from dev-machine success
Jury asks for a cost/scale number not in this document	Low	Medium — looks unprepared live	`SCALABILITY.md` covers GPU sizing, bandwidth, storage, and rollout cost — reference it directly in the presentation, don't re-derive live
Real camera GIS/department metadata from [illegible] against the live sandbox	Medium	Low-Medium — cosmetic on the map, not a pipeline failure	Script output is explicitly flagged as needing manual review (README), not trusted as ground truth by default